

MedTrace

PS-03-S2 — Predicting Medicine Stockouts Before They Happen

Hackathon Product Blueprint | Beginner-Friendly Build Document

One-line idea: MedTrace predicts medicine shortages, checks whether the reported stock data can be trusted, finds nearby safe surplus, and recommends an officer-approved rescue action before patients are turned away.

1. Problem Statement

Official problem ID: PS-03-S2 — Predicting Medicine Stockouts Before They Happen.

Primary health centres can run out of essential medicines such as insulin, ORS, anti-tuberculosis drugs, antivenom, and antibiotics. Often the shortage is discovered only when a patient arrives and is told that the medicine is unavailable.

The challenge is not only predicting low stock. A facility may appear to have zero consumption because it stopped reporting, even though physical stock exists. Another facility may have excess or near-expiry medicine that could be transferred safely. Repeated shortages may point to procurement or distribution problems rather than a one-time forecasting error.

Simple problem sentence: Health officers need a way to identify likely medicine shortages early, distinguish real shortages from bad reporting, and take the best practical action before treatment is interrupted.

2. Why We Chose It

- **Easy to understand:** Every judge understands the harm when a patient cannot get an essential medicine.
- **High social impact:** It supports treatment continuity for vulnerable patients.
- **Technically strong:** It combines forecasting, data-quality checks, optimisation, explainability, and human approval.
- **Buildable:** The hackathon MVP can use realistic fictional data; no hospital hardware, paid API, or government data access is required.
- **Great demo:** A facility will run out in two days; the system finds nearby safe surplus and shows a rescue plan.
- **Scalable:** Start with 20 demo facilities and later integrate authorised historical or live records.

3. Hidden Problem We Solve

The visible problem is: **“A health centre has no medicine.”** The hidden problem is that medicine, information, and decisions are disconnected.

Hidden case	What a basic dashboard sees	What MedTrace does
Real stockout vs reporting gap	“Stock = 0” or “usage = 0”	Checks whether the report is late/stale before recommending a transfer
Phantom stock	System says stock exists	Flags unreliable record and asks for physical verification
Hidden surplus	One PHC is critical	Finds another nearby PHC with safe transferable surplus
Near-expiry stock	Stock is available but may be wasted	Matches usable near-expiry stock with a facility that can consume it
Demand shock	Sudden high use	Separates likely demand surge from poor reporting or reorder delay

Hidden case	What a basic dashboard sees	What MedTrace does
Chronic stockout	Repeated low-stock alert	Shows likely procurement/distribution root cause
Unfair transfer	Nearest donor is chosen	Protects remote/donor facilities with minimum safety stock

4. Final Solution

Product name: MedTrace

Product definition: An explainable medicine-access rescue network that forecasts stockout risk, checks stock-data trustworthiness, detects likely causes, finds safe nearby surplus, and recommends an officer-approved transfer plan.

What we are NOT building: A generic inventory dashboard, a chatbot, an automatic medicine-transfer system, or a black-box AI that makes medical decisions.

What we ARE building: A decision-support system that tells a health officer what to verify and what safe action is recommended.

5. Core Workflow

Health officer uploads data or clicks “Load Demo Data” → MedTrace validates data → calculates stock and forecast → checks Stock Truth Score → identifies stockout risk → finds safe surplus → recommends transfer or verification → officer approves/rejects → feedback is saved.

Workflow diagram:

```

Inventory + issue data + expiry + facility location
↓
Demand forecast and days-of-stock calculation
↓
Reporting-gap and Stock Truth Score
↓
Stockout risk classification
↓
Surplus and expiry matching
↓
Fair rescue-transfer recommendation
↓
Officer approval and feedback

```

6. Main Modules

#	Module	What it must do	MVP priority
1	Demo Data Generator	Create 20 fictional PHCs, 8 medicines, 90 days of records, including hidden cases	Must
2	CSV Upload + Validator	Validate dates, stock quantities, duplicates, missing reports, and stock-balance mismatch	Must
3	Inventory Status Engine	Calculate current stock, daily demand, days remaining, safety stock, status	Must
4	Demand Forecast Engine	Forecast next 7-day demand using explainable weighted averages	Must
5	Stock Truth / Reporting-Gap Engine	Score whether records are current and trustworthy	Must / Core novelty
6	Stockout Risk Engine	Combine stock, forecast, and trust score into risk and confidence	Must
7	Surplus + Expiry Engine	Find safe donors and identify near-expiry transfer opportunities	Must
8	Rescue Transfer Optimiser	Recommend donor, recipient, quantity, reason, distance, and warnings	Must / Wow
9	Root-Cause Engine	Detect repeat risks and likely procurement/distribution issues	Advanced

#	Module	What it must do	MVP priority
10	Dashboard + Feedback	Show rescue queue, details, simulator, approval and feedback	Must

7. Key Novelty Features

- **Stock Truth Score:** Before trusting “stock = 0,” the system measures whether the report is recent, complete, and consistent.
- **Reporting-gap detector:** It detects when a missing report may be mistaken for a real shortage. The action becomes “verify physical stock first,” not “transfer immediately.”
- **Rescue optimiser:** It does not stop at an alert. It recommends where to transfer stock from, how much to send, and why it is safe.
- **Expiry-to-need matching:** It can recommend moving usable near-expiry stock to a high-demand facility, reducing both shortage risk and wastage.
- **Fairness rule:** It will not empty a donor or remote facility below protected safety stock just because it is close by.
- **Counterfactual simulator:** If an officer confirms an updated physical count, the system recalculates the risk and may cancel an unnecessary transfer.
- **Chronic-risk explanation:** Repeated stockout risk can be labelled as a likely procurement, reorder-point, demand, reporting, or distribution problem—not merely “bad forecast.”

8. Example Demo Story

Case: PHC A may run out of insulin.

- PHC A has 24 insulin vials.
- Forecast demand is 12 vials/day.
- Estimated stockout is in 2 days.
- Stock Truth Score is 92/100 because reporting is current and the recent physical count is consistent.
- PHC B, 8 km away, has a safe surplus of 160 vials after retaining its own safety stock.
- The district warehouse can replenish but only in 6 days.

MedTrace recommendation: Transfer 84 vials from PHC B to PHC A today. This protects PHC A for approximately 7 days, keeps PHC B above its protected safety stock, and is faster than warehouse delivery.

Counterfactual “wow” moment: The officer marks “physical count completed” and enters that PHC A actually has 100 unreported vials. MedTrace updates the Stock Truth Score and cancels the transfer recommendation. This proves the system is fair and does not move medicine blindly.

9. Architecture and Tech Stack

Recommended beginner-friendly stack:

Layer	Technology	Why
Project home	VS Code + GitHub	All final code stays in one controlled repository
Main AI coding helper	Antigravity	Generate/edit modules and run project tasks inside the coding workspace
Planning/debugging helper	Claude or ChatGPT	Explain errors, review code, improve prompts and pitch
Optional UI helper	Lovable	Create UI layout/mockup only; export/sync to GitHub then continue in VS Code

Layer	Technology	Why
Frontend	React + Vite + Axios + Plotly	Professional dashboard, API calls, trend charts
Backend	Python FastAPI	Simple APIs and clear automatic documentation at /docs
Data processing	Pandas + NumPy	CSV processing and calculations
Forecasting	Weighted moving average / simple rules	Explainable and realistic for a hackathon
Database	SQLite	Free, local, simple for feedback/recommendations
Optional optimiser	Simple deterministic score; OR-Tools later	Avoid over-engineering MVP

10. Tool Rules

Use VS Code and GitHub as the final project home. Do not create separate, conflicting versions in multiple AI platforms.

Tool	Use it for	Avoid using it for
Antigravity	Primary coding: backend files, React components, API integration, bug fixing	Project storage or uncontrolled "build everything" prompts
Claude / ChatGPT	Understanding concepts, reviewing errors, writing small prompts, presentation content	Running a second version of the app
Lovable	Optional dashboard visual design and React mock UI	Core backend/forecast/optimiser logic
Emergent	Emergency backup prototype only if main setup fails	Parallel primary development
VS Code	Run frontend/backend, edit files, integrate modules	Nothing replaces this
GitHub	Backup after every working feature	A substitute for testing locally

11. API Plan

Endpoint	Purpose
GET /health	Check whether backend is running
POST /generate-demo-data	Generate/load fictional PHC and medicine datasets
POST /upload-inventory	Upload and validate inventory CSV
GET /dashboard-summary	Return dashboard numbers
GET /facilities	List facilities
GET /facilities/{facility_id}	Return facility summary
GET /facilities/{facility_id}/medicines/{medicine_id}	Return stock, demand, forecast, truth score and trend data
GET /rescue-queue	Rank stockout-risk cases
GET /rescue-cases/{case_id}	Return complete evidence and recommended action
POST /recommend-transfer	Return donor-recipient transfer recommendation
POST /simulate-verification	Recalculate after physical-count/report verification
POST /feedback	Store officer decision
GET /root-causes	Show chronic risk explanations

12. Final Project Structure

Create one project folder named **medtrace** in VS Code:

```
medtrace/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── synthetic_data.py
│   │   ├── validator.py
│   │   ├── inventory_engine.py
│   │   ├── forecast_engine.py
│   │   ├── stock_truth_engine.py
│   │   ├── stockout_risk_engine.py
│   │   ├── surplus_engine.py
│   │   ├── transfer_optimizer.py
│   │   ├── root_cause_engine.py
│   │   ├── database.py
│   └── data/
│       ├── facilities.csv
│       ├── medicine_config.csv
│       ├── inventory_records.csv
│       └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   └── services/api.js
│   ├── .env
│   ├── package.json
│   ├── README.md
│   └── .gitignore
```

13. Build Order

Phase	Build this	Do NOT build yet
Phase 1: Foundation	Backend /health, demo data, inventory status, dashboard summary	Frontend, login, maps, AI chatbot
Phase 2: Core novelty	Forecast, Stock Truth Score, reporting-gap detection, stockout risk	Complex ML/deep learning
Phase 3: Rescue action	Surplus calculation, expiry check, transfer recommendation, simulator	Automatic medicine transfer
Phase 4: Frontend	Dashboard, rescue queue, facility page, transfer plan, feedback	Mobile app, payments, chatbots
Phase 5: Polish	Root cause page, presentation, tests, backup demo video	New features that do not appear in demo

14. 48-Hour MVP Plan

- **Hours 0–2:** Choose final scope, create GitHub repository, create VS Code project, set up Python and Node.
- **Hours 2–6:** Build FastAPI /health endpoint and generate fictional CSV data.
- **Hours 6–12:** Build inventory status, simple demand forecast, and stockout risk calculation. Test in FastAPI /docs.
- **Hours 12–18:** Build Stock Truth Score/reporting-gap detector and create three polished hidden-case scenarios.
- **Hours 18–24:** Build surplus engine and transfer recommendation. Add counterfactual simulation endpoint.
- **Hours 24–32:** Build React dashboard, rescue queue, and facility detail view.
- **Hours 32–38:** Connect frontend to backend. Add charts, rescue recommendation, and feedback.
- **Hours 38–48:** Fix bugs, improve UI wording, rehearse demo, create slides, and record a backup video.

15. Exact Starter Prompt for Antigravity

Paste this in Antigravity while your **medtrace** folder is open in VS Code. Build only Phase 1 first:

I am a beginner building a hackathon project called MedTrace. MedTrace predicts medicine stockouts, checks whether stock data is trustworthy, finds nearby safe surplus, and recommends an officer-approved rescue transfer. Use Python FastAPI backend, local CSV demo data, and no paid APIs. Build Phase 1 only: 1. Create backend project structure. 2. Add GET /health returning {"status": "ok"}. 3. Create synthetic_data.py that generates fictional data for 20 PHCs, 8 medicines, and 90 days. 4. Add POST /generate-demo-data that creates/loads the CSV data. 5. Create inventory_engine.py that calculates current stock, average daily demand, days of stock remaining, and stock status. 6. Add GET /dashboard-summary. 7. Add requirements.txt and simple README instructions. 8. Run the backend and fix errors. Do not build frontend, chatbots, login, maps, deployment, deep learning, or complex optimisation yet. After completing, list changed files, exact terminal commands, tests performed, and known limitations.

16. Important Safety and Truth Rules

- Use fictional/anonymised data for the hackathon unless authorised data is available.
- Say “possible stockout,” “verification required,” and “officer approval required.”
- Do not claim a diagnosis, medical advice, or automatic transfer authority.
- Do not state real accuracy figures unless you evaluate them on clearly labelled data.
- Do not claim that the prototype has already saved patients or reduced stockouts in the real world.
- In a real deployment, transfers must follow applicable medicine quality, transport, cold-chain, expiry, and government-policy requirements.

17. Team Explanation Script

“Most teams may build a normal medicine-stock dashboard that says a medicine will finish in three days. We are building something deeper. A stockout can be real, or it can be caused by a reporting gap. Medicine can be available at another PHC but invisible to the facility that needs it. Some stock may expire unused while another PHC runs out. MedTrace predicts risk, checks whether the data can be trusted, finds safe surplus, and recommends an officer-approved rescue plan. So we are not just predicting shortages; we are uncovering hidden medicine-access failures before patients are affected.”

18. Final Checklist

- One working synthetic dataset with at least 3 strong hidden-case stories.
- Working backend shown in FastAPI Swagger documentation.
- Working dashboard with a rescue queue.
- Stock Truth Score displayed clearly.
- One transfer recommendation with donor safety proof.
- One counterfactual simulation: physical verification changes the recommendation.
- Clear “Officer approval required” disclaimer.
- Slides and recorded backup demo.